

King Saud University

College of Computer and Information Sciences

Department of Computer Sciences

CSC 227: Operating Systems

Second Semester 1447H

Multithreaded CPU Scheduling Simulator

Course Project Report | Section 56301

Name	Student ID
Hawra Alhammad (Leader)	445200321
Aroub Alswayyed	445202232
Joud BinFaris	445201208
Bodur Albarqi	445201994
Rawan Alshammari	444200348

1. Task Distribution

Name	Coding Task
Joud BinFaris	PCB class, FileReaderThread (Thread 1), job.txt parser, Job Queue setup
Bodur Albarqi	MemoryManager class (Thread 2), memory tracking, Ready Queue management, thread synchronization
Hawra Alhammad	SJFScheduler class, Gantt chart, results table, performance metrics
Aroub Alswayyed	RoundRobinScheduler class, Gantt chart, results table, performance metrics
Rawan Alshammari	PriorityScheduler class with aging and starvation detection, Main class with algorithm selection menu

2. Software and Hardware Tools

Software Tools

- Programming Language: Java (JDK 23)

- IDE: Visual Studio Code with Extension Pack for Java
- Version Control: Git and GitHub for team collaboration

Hardware Tools

- Personal laptops running Microsoft Windows 10 and Windows 11

3. Multithreading and Synchronization

How Multithreading Enhances Performance

This application uses two independent threads alongside the main thread to simulate a real operating system environment. Thread 1 is responsible for reading the job.txt file and creating PCB objects for each process, which it then adds to the shared Job Queue. Thread 2 acts as the memory manager, continuously monitoring the Job Queue and transferring processes to the Ready Queue only when sufficient memory is available.

Without multithreading, these operations would execute sequentially: the file reader would complete first, followed by the memory manager, and then the scheduling algorithm. By using separate threads, Thread 1 and Thread 2 can run concurrently, allowing file reading and memory management to overlap. This mirrors how real operating systems handle process management in parallel and improves the responsiveness and efficiency of the simulation.

Additionally, multithreading allows the main thread to remain focused on the scheduling algorithm while the other threads handle data loading and memory management independently. This separation of responsibilities makes the application more structured, maintainable, and closer to real OS behavior.

Synchronization Mechanisms

Synchronization is required in this application because multiple threads share access to the same data structures. Specifically, Thread 1 and Thread 2 both access the Job Queue, and Thread 2 and the main thread both access the Ready Queue. Without synchronization, two threads could simultaneously read from or write to these queues, leading to race conditions and data corruption.

We used Java's built-in synchronized blocks to protect all shared queue operations. The synchronized keyword ensures that only one thread can access a critical section at a time by acquiring a lock on the shared

object. If a second thread tries to access the same synchronized block, it is forced to wait until the first thread releases the lock.

We chose synchronized blocks over other mechanisms such as ReentrantLock or concurrent data structures because it is simpler to implement, sufficient for the level of concurrency in this project, and does not require additional imports or configuration. It is also well-suited for short, bounded critical sections such as adding or removing items from a queue.

4. Preferred Output Format: Gantt Chart vs Table

We chose to include both the Gantt chart and the results table in our program output, as each format serves a distinct and complementary purpose.

The Gantt chart is our preferred format for visualizing the scheduling behavior. It provides a clear timeline of process execution, showing exactly which process ran at each moment, in what order, and for how long. This is particularly valuable for the Round Robin algorithm, where processes alternate rapidly and a table alone would not make the execution order obvious. The Gantt chart also makes it easy to spot patterns such as starvation, where a process appears very late or not at all in the timeline.

The results table complements the Gantt chart by providing precise numerical data for every process. It includes the process ID, burst time, start time, termination time, waiting time, and turnaround time, along with the average waiting and turnaround times across all processes. This format enables direct quantitative comparison between the three algorithms.

Using both formats together gives a complete picture: the Gantt chart answers the question of when and in what order processes executed, while the table answers the question of how efficient the algorithm was. Neither format alone is sufficient to fully evaluate a scheduling algorithm, which is why we chose to include both.

5. Program Output Screenshots

Input File (job.txt)

The following input file was used for all three algorithm runs:

- P1: Burst = 25ms, Priority = 4, Memory = 500MB
- P2: Burst = 13ms, Priority = 3, Memory = 700MB
- P3: Burst = 20ms, Priority = 3, Memory = 100MB
- P4: Burst = 13ms, Priority = 2, Memory = 200MB
- P5: Burst = 10ms, Priority = 1, Memory = 150MB
- P6: Burst = 30ms, Priority = 5, Memory = 180MB
- P7: Burst = 8ms, Priority = 2, Memory = 120MB
- P8: Burst = 22ms, Priority = 6, Memory = 90MB
- P9: Burst = 15ms, Priority = 1, Memory = 100MB
- P10: Burst = 12ms, Priority = 4, Memory = 80MB

Thread 1 and Thread 2

```
Thread 1: reading file src/proj/job.txt
Thread 2 started: Memory Manager is running...

Thread 1: Loaded PCB ID=1, Burst=25ms, Priority=4, Memory=500MB

Admitted: P1 | Memory Used: 500/2048 MB
Admitted: P2 | Memory Used: 1200/2048 MB
Thread 1: Loaded PCB ID=2, Burst=13ms, Priority=3, Memory=700MB

Thread 1: Loaded PCB ID=3, Burst=20ms, Priority=3, Memory=100MB

Admitted: P3 | Memory Used: 1300/2048 MB
Thread 1: Loaded PCB ID=4, Burst=13ms, Priority=2, Memory=200MB

Thread 1: Loaded PCB ID=5, Burst=10ms, Priority=1, Memory=150MB

Thread 1: Loaded PCB ID=6, Burst=30ms, Priority=5, Memory=180MB

Thread 1: Loaded PCB ID=7, Burst=8ms, Priority=2, Memory=120MB

Thread 1: Loaded PCB ID=8, Burst=22ms, Priority=6, Memory=90MB

Thread 1: Loaded PCB ID=9, Burst=15ms, Priority=1, Memory=100MB

Thread 1: Loaded PCB ID=10, Burst=12ms, Priority=4, Memory=80MB

Admitted: P4 | Memory Used: 1500/2048 MB
Admitted: P5 | Memory Used: 1650/2048 MB
Thread 1: finished loading processes into Job Queue.
Admitted: P6 | Memory Used: 1830/2048 MB
Admitted: P7 | Memory Used: 1950/2048 MB
Admitted: P8 | Memory Used: 2040/2048 MB
Waiting: P9 | Needs: 100 MB | Available: 8 MB
```

```
Processes inside Ready Queue:
Waiting: P10 | Needs: 80 MB | Available: 8 MB
PCB ID=1, Burst=25ms, Priority=4, Memory=500MB

PCB ID=2, Burst=13ms, Priority=3, Memory=700MB

PCB ID=3, Burst=20ms, Priority=3, Memory=100MB

PCB ID=4, Burst=13ms, Priority=2, Memory=200MB

PCB ID=5, Burst=10ms, Priority=1, Memory=150MB

PCB ID=6, Burst=30ms, Priority=5, Memory=180MB

PCB ID=7, Burst=8ms, Priority=2, Memory=120MB

PCB ID=8, Burst=22ms, Priority=6, Memory=90MB
```

Algorithm 1: Shortest Job First (SJF)

```

Choose a scheduling algorithm:
1. Shortest Job First (SJF)
2. Round Robin (q = 5 ms)
3. Priority Scheduling (with Aging)
Enter your choice (1-3): 1

=====
Shortest Job First (SJF) Scheduler
=====

Freed: P7 | Memory Used: 1920/2048 MB
Admitted: P9 | Memory Used: 2020/2048 MB
Freed: P5 | Memory Used: 1870/2048 MB
Admitted: P10 | Memory Used: 1950/2048 MB
Freed: P10 | Memory Used: 1870/2048 MB
Freed: P2 | Memory Used: 1170/2048 MB
Freed: P4 | Memory Used: 970/2048 MB
Freed: P9 | Memory Used: 870/2048 MB
Freed: P3 | Memory Used: 770/2048 MB
Freed: P8 | Memory Used: 680/2048 MB
Freed: P1 | Memory Used: 180/2048 MB
Freed: P6 | Memory Used: 0/2048 MB
Gantt Chart:
| P7(8->0) | P5(10->0) | P10(12->0) | P2(13->0) | P4(13->0) | P9(15->0) | P3(20->0) | P8(22->0) | P1(25->0) | P6(30->0) |
0          8          18          30          43          56          71          91          113          138          168

Results Table:
-----
Process ID | Burst Time | Start Time | Termination Time | Waiting Time | Turnaround Time
-----
1          | 25         | 113        | 138               | 113          | 138
2          | 13         | 30         | 43                | 30           | 43
3          | 20         | 71         | 91                | 71           | 91
4          | 13         | 43         | 56                | 43           | 56
5          | 10         | 8          | 18                | 8            | 18
6          | 30         | 138        | 168               | 138          | 168
7          | 8          | 0          | 8                 | 0            | 8
8          | 22         | 91         | 113               | 91           | 113
9          | 15         | 56         | 71                | 56           | 71
10         | 12         | 18         | 30                | 18           | 30
-----

Average Waiting Time : 56.80 ms
Average Turnaround Time : 73.60 ms

```

Algorithm 2: Round Robin (q = 5ms)

```
3. Priority Scheduling (with Aging)
Enter your choice (1-3): 2

=====
Round Robin Scheduler (q = 5 ms)
=====

Freed: P5 | Memory Used: 1890/2048 MB
Admitted: P9 | Memory Used: 1990/2048 MB
Freed: P7 | Memory Used: 1870/2048 MB
Admitted: P10 | Memory Used: 1950/2048 MB
Freed: P2 | Memory Used: 1250/2048 MB
Freed: P4 | Memory Used: 1050/2048 MB
Freed: P3 | Memory Used: 950/2048 MB
Freed: P1 | Memory Used: 450/2048 MB
Freed: P9 | Memory Used: 350/2048 MB
Freed: P10 | Memory Used: 270/2048 MB
Freed: P8 | Memory Used: 180/2048 MB
Freed: P6 | Memory Used: 0/2048 MB
Gantt Chart:
| P1(25->20) | P2(13->8) | P3(20->15) | P4(13->8) | P5(10->5) | P6(30->25) | P7(8->3) | P8(22->17) | P1(20->15) | P2(8->3) | P3(15->10) | P4(8->3) | P5(5->0) | P6(25->20) | P7(3->0) | P8(17->12) | P1(15->10) | P2(3->0) | P3(10->5) | P4(3->0) | P9(15->10) | P6(20->15) | P10(12->7) | P8(12->7) | P1(10->5) | P3(5->0) | P9(10->5) | P6(15->10) | P10(7->2) | P8(7->2) | P1(5->0) | P9(5->0) | P6(10->5) | P10(2->0) | P8(2->0) | P6(5->0) |
0      5      10     15     20     25     30     35     40     45     50     55     60     65     70
73     78     83     86     91     94     99     104    109    114    119    124    129    134    139
144    149    154    159    161    163    168

Results Table:
-----
Process ID  Burst Time  Start Time  Termination Time  Waiting Time  Turnaround Time
-----
1           25         0           149              124          149
2           13         5           86              73           86
3           20        10          124             104          124
4           13        15           94              81           94
5           10        20           65              55           65
6           30        25          168             138          168
7            8        30           73              65           73
8           22        35          163             141          163
9           15        94          154             139          154
10          12       104          161             149          161
-----

Average Waiting Time : 106.90 ms
Average Turnaround Time : 123.70 ms
```

Algorithm 3: Priority Scheduling with Aging

```

Choose a scheduling algorithm:
1. Shortest Job First (SJF)
2. Round Robin (q = 5 ms)
3. Priority Scheduling (with Aging)
Enter your choice (1-3): 3

=====
Priority Scheduler (Non-Preemptive)
=====

Freed: P5 | Memory Used: 1890/2048 MB
Admitted: P9 | Memory Used: 1990/2048 MB
Freed: P4 | Memory Used: 1790/2048 MB
Admitted: P10 | Memory Used: 1870/2048 MB
Freed: P7 | Memory Used: 1750/2048 MB
Freed: P2 | Memory Used: 1050/2048 MB
Freed: P3 | Memory Used: 950/2048 MB
Freed: P9 | Memory Used: 850/2048 MB
Freed: P1 | Memory Used: 350/2048 MB
Freed: P6 | Memory Used: 170/2048 MB
Freed: P8 | Memory Used: 80/2048 MB
Freed: P10 | Memory Used: 0/2048 MB
Gantt Chart:
| P5(10->0) | P4(13->0) | P7(8->0) | P2(13->0) | P3(20->0) | P9(15->0) | P1(25->0) | P6(30->0) | P8(22->0) | P10(12->0) |
0          10          23          31          44          64          79          104          134          156          168

Results Table:
-----
Process ID | Burst Time | Start Time | Termination Time | Waiting Time | Turnaround Time
-----
1          | 25         | 79         | 104              | 79          | 104
2          | 13         | 31         | 44              | 31          | 44
3          | 20         | 44         | 64              | 44          | 64
4          | 13         | 10         | 23              | 10          | 23
5          | 10         | 0          | 10              | 0           | 10
6          | 30         | 104        | 134             | 104         | 134
7          | 8          | 23         | 31              | 23          | 31
8          | 22         | 134        | 156             | 134         | 156
9          | 15         | 64         | 79              | 64          | 79
10         | 12         | 156        | 168             | 156         | 168
-----

```

Average Waiting Time : 64.50 ms
Average Turnaround Time : 81.30 ms

Starvation Report:

Process P1 suffered from starvation. Aging was applied.
Process P3 suffered from starvation. Aging was applied.
Process P6 suffered from starvation. Aging was applied.
Process P8 suffered from starvation. Aging was applied.
Process P9 suffered from starvation. Aging was applied.
Process P10 suffered from starvation. Aging was applied.

6. Additional Functionalities

Defensive Copying in All Schedulers

Each scheduler creates a private copy of the ready queue upon construction rather than operating directly on the shared queue. This ensures that the original ready queue managed by Thread 2 is never modified or disrupted by sorting, polling, or re-ordering operations performed during scheduling. This design decision protects data integrity across threads and makes each scheduler self-contained and independently testable using hardcoded process lists without requiring the full threading infrastructure.